ConfigStream

Scalable Configuration Change Storage and Streaming Analytics

Phạm Công Hoàng Lê Tiến Hợp
Mai Vũ Hoàng Nguyễn Đức Dũng
Group 4 · Big Data Storage & Processing

First milestone checkpoint · 26 October 2026

The problem: configuration changes over time

Router configuration: version 10

```
hostname router-001
logging enabled
telnet disabled
```

Version 11

```
hostname router-001
logging disabled
telnet enabled
```

What changed, when, and on which asset?

A current configuration file alone cannot answer all three.

Illustrative input from the executable demo: processing/demo.py.

What ConfigStream provides



A searchable history of configuration changes

For configuration histories, one asset ID links versions, differences and alerts.

Implemented contracts: schemas/; processing/; api/routes/documents.py.

Four datasets, four meanings of change

Terraform / TerraDS: versions

Infrastructure code stored in Git.

v1 → **v2**: which lines changed?

RPKI: observed daily states

Observed route-authorization records.

Day 1 → **day 2**: which members differ?

RIS: routing events

BGP announcements and withdrawals.

Announce → **withdraw**: what happened?

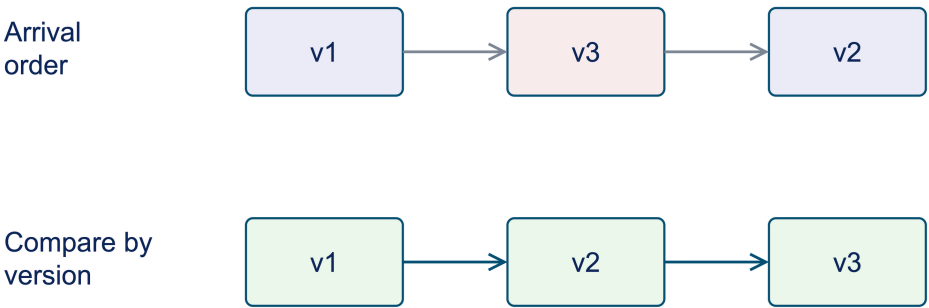
DNS: measurements

Recorded DNS query and response data.

Name → **IP**: what was observed, when?

Conceptual examples. Sources and accepted scopes: frontend/public/research/v1/catalog.json.

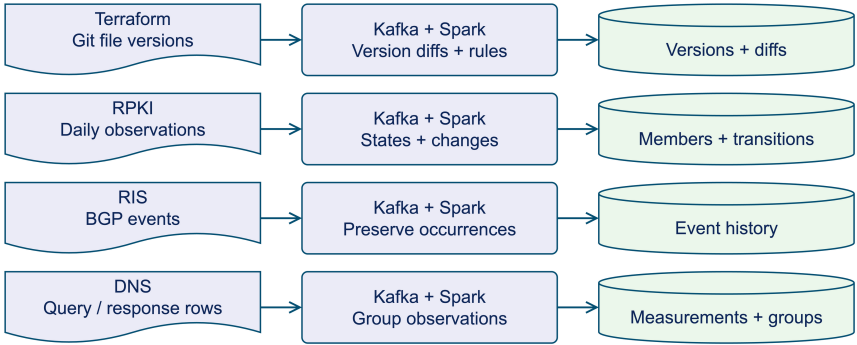
Configuration example: handling late arrivals



Compare **v1** → **v2** and **v2** → **v3**, once both endpoints are available.

processing/state.py; spark/streaming/stateful.py. Online repair is bounded by retained state.

System architecture: four separate processing paths



Kafka + Spark + MinIO (Parquet), with separate schemas and processing.
Config search: Elasticsearch + FastAPI. Dashboard: accepted results from all four.

spark/streaming/; spark/rpki_stream.py; spark/ris.py; spark/dns.py; domain architecture docs.

Prepared data: accepted scope for each dataset

Dataset	Accepted amount	Selected scope
Terraform	**10,188** Git versions	6,505 file histories / 375 repositories
RPKI	**14** complete daily observations	ripencc.tal, 1–14 Sep 2026
RIS	**4,725,156** event occurrences	rrc00, 1 Sep 2026, 00:00–01:00 UTC
DNS	**2,234,863** raw measurements	Two .li source objects, 1–2 Sep 2026

Controlled synthetic inputs also test retries and late arrivals.
Source manifests and result receipts are versioned in Git.

Accepted evidence: frontend/public/research/v1/catalog.json and its linked validation artifacts. Units differ; do not sum the rows.

Implemented: differences and rule alerts

Diff: version 10 → 11

```
- logging enabled
- telnet disabled
+ logging disabled
+ telnet enabled
```

Rules detect

Logging disabled
High severity

Telnet enabled
Critical severity

Repeated delivery must not invent another logical change.

processing/demo.py; processing/diff.py; rules/default.yaml; tests/unit/test_rules.py.

Executable code is available in Git

Repository

`generator/`	input data
`processing/`	diff and rules
`spark/`	streaming jobs
`api/`	read API
`frontend/`	dashboard

Small demo, no services required

```
make setup
make processing-demo
```

178 core tests passed
State logic, diffs and rules
Rechecked on 25 October 2026

github.com/kongwoang/ConfigStream; Makefile; tests/unit/test_state_logic.py, test_diff.py, test_rules.py.

A complete local replay has been recorded

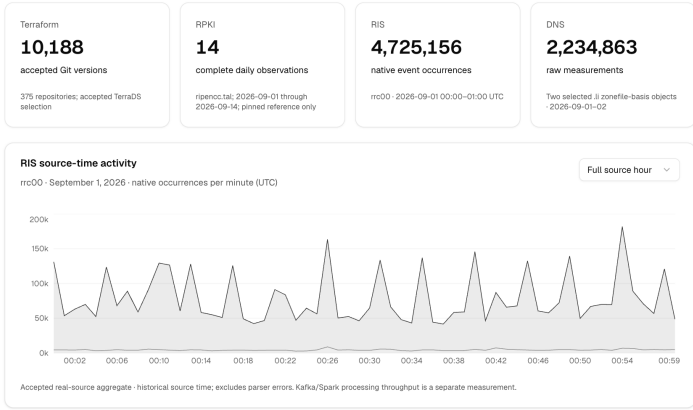


29.4 MB of raw and processed Parquet committed.

This demonstrates the integrated path on the selected local workload.

Recorded acceptance: docs/phase12-acceptance.md. 29,402,997 bytes; not a new replay or a scale benchmark.

Dashboard preview



Actual frontend captured on 25 October 2026. This overview displays the accepted evidence bundle, not a live replay.

Checkpoint readiness

Requirement	What the group can show
Git + executable code	Repository, runnable demo and automated tests
Data ready	Terraform, RPKI, RIS, DNS and source manifests
Architecture ready	Four processing paths, shared storage and dashboard

The core workflow is implemented and demonstrated locally

Next milestone

1. **Rehearse the integrated demo**
   Start the services, send a change, inspect the stored result.
2. **Evaluate larger workloads**
   Measure throughput, latency and storage with clear units.
3. **Strengthen recovery evidence**
   Repeat failure and restart checks for the latest revision.

Current evidence is local / single-node; distributed scale and HA remain open.

Thank you — questions?